HATHOR-CORE · MASTER-BOOK

Type Hints & Static Typing

How optional type annotations turn Python's flexibility into a checkable contract — and why a codebase the size of a node depends on it.

SUBJECT	hathor-core · Part I · Track A (programming concepts)
CHAPTER	05 · Foundations · Concepts
BRANCH	study/hathor-core-studies
EDITION	2026 · v1

Dynamic vs static typing · Type hints · mypy · Optional / Union · Generics · TypeVar · Protocols (structural typing) · Pydantic

Table of Contents

Chapter 5 — Type Hints & Static Typing

What you'll learn in this chapter

- The difference between **dynamic** and **static** typing, and the specific cost dynamic typing imposes on a large codebase.
- **Type hints**: the annotation syntax (`x: int`, `-> str`, `list[int]`, `int | None`), and the surprising fact that Python mostly ignores them at run time.
- The **static type checker** (mypy): a tool that proves type-correctness without running the code.
- The working vocabulary: `Optional` / `Union`, `Any`, generics and `TypeVar`, and **Protocols** — duck typing made checkable.
- Why a node-sized project pays the annotation cost, and the honest trade-offs.
- A **bridge** to typing in `hathor-core` (`types.py`, the mypy toolchain, the zope plugin, and Pydantic — where types do act at run time).

This is the last of the programming-concept chapters. It is shorter than the others because it adds a discipline on top of code you already understand, rather than a new runtime mechanism. But it is the discipline that makes a 60,000-line codebase navigable: when every function says what it takes and returns, and a tool checks that those promises hold, reading and changing the code becomes far safer. After this, Track B turns from how the code is built to what it is about — blockchains, UTXOs, the DAG.

5.1 Dynamic typing and its cost at scale

Python is **dynamically typed**[1]: a variable has no fixed type, and the type of a value is checked only when the code actually runs. This is why you can write `x = 5` and later `x = "hello"` with no complaint, and why a function happily accepts any argument until something inside it fails. It makes Python fast to write and flexible — and it has a precise cost that grows with the size of the program.

The cost is that **type errors stay hidden until the exact line runs**. Consider:

```python
def total_fees(transactions):
    return sum(t.fee for t in transactions)
```

What is `transactions`? A list? Of what? What is `t.fee` — an int, a float, an object? Nothing here says. If a caller passes the wrong thing, you find out only when this line executes — perhaps in production, perhaps months later, on a rare branch. In a small script that is harmless. In a node with thousands of functions calling each other, "what type flows through here?" becomes a question you cannot answer by reading — only by running, and only for the paths you happen to exercise. Whole categories of bug (passing `None` where an object is expected, a string where a number is expected, a misspelled attribute) lurk until run time.

Type hints exist to drag those errors forward — out of "discovered at run time, maybe" and into "caught before the code ever runs."

5.2 Type hints: annotations that document and check

A **type hint** (or **annotation**[2]) is a label you attach to a variable, parameter, or return value, declaring the type it is expected to hold. The syntax is light:

```
def total_fees(transactions: list[Transaction]) -> int:   # takes a list of Transaction, returns
    return sum(t.fee for t in transactions)

count: int = 0                        # a variable annotation
name: str = "node-1"
```

Read `param: Type` as "this parameter should be a `Type`," and `-> Type` as "this function returns a `Type`." Containers are spelled with brackets: `list[int]` (a list of ints), `dict[str, int]` (a dict from str to int), `tuple[int, int]` (a pair of ints).

Here is the fact that surprises everyone: **Python does not enforce these at run time.** The annotations are, to the interpreter, almost decoration — it will not stop you calling `total_fees("oops")`. So what are they for? Two audiences:

1. **Humans.** The signature now documents itself, and — unlike a comment — the documentation lives in the code where it can be checked.
2. **Tools.** A separate program reads the annotations and verifies the code obeys them. That program is the static type checker, and it is where the value comes from.

> **A note on "optional."** Python's typing is gradual[3]: you can annotate as much or as little as you like, mixing typed and untyped code. You can add hints to one function and leave the rest bare. This lets a codebase adopt typing incrementally rather than all at once.

5.3 The static type checker (mypy)

A **static type checker** reads your annotated code and proves it type-consistent **without executing it** — analysing the source as text, following types from where they're produced to where they're used, and flagging any mismatch. "Static"[4] means exactly this: analysis done ahead of time, on the code at rest, as opposed to dynamic checks that happen while it runs. The standard checker for Python, and the one `hathor-core` uses, is **mypy**[5].

Give mypy this:

```python
def greet(name: str) -> str:
    return "hi " + name

greet(42)        # we pass an int where a str is required
```

...and without ever running the program, mypy reports:

```
error: Argument 1 to "greet" has incompatible type "int"; expected "str"
```

That error would otherwise have waited for the line to execute. Multiply this across a whole codebase and you have a net that catches a large class of bugs the moment you save the file — typically wired into the editor and the continuous-integration[6] pipeline so nothing untyped or mistyped slips through. This is **static analysis**[7]: learning facts about a program from its source without running it, of which type-checking is the most common form.

The mental model: hints are claims; mypy is the proof-checker. Writing `-> str` claims "this returns a string"; mypy verifies the claim against the actual `return` statements, and verifies every caller uses the result as a string.

5.4 The vocabulary of types

A handful of constructs cover most annotations you'll read in the codebase.

`Optional` / `T | None`. Code that may return "a value or nothing" is everywhere (a lookup that might miss). The type is "T or None," written `int | None` (modern syntax) or `Optional[int]` (older, identical meaning). The payoff: mypy then forces callers to handle the `None` case, eliminating the classic "called a method on None" crash.

```python
def find_height(block_hash: str) -> int | None:    # an int, or None if not found
    ...

h = find_height(x)
print(h + 1)          # mypy ERROR: h might be None — handle it first
```

`Union` and `Any`. `Union[A, B]` (or `A | B`) means "either type." `Any`[8] is the escape hatch: it means "any type, stop checking here." `Any` is occasionally necessary but is a hole in the net — every value flowing through `Any` is unchecked — so good codebases minimize it.

Generics and `TypeVar`. Some functions work for any type while preserving it: a function returning the first element of a list returns whatever the list holds. A **generic**[9] expresses that with a **type variable**[10]:

```python
from typing import TypeVar

T = TypeVar("T")

def first(items: list[T]) -> T:      # returns the SAME type the list contains
    return items[0]

first([1, 2, 3])        # mypy knows this is an int
first(["a", "b"])       # ...and this is a str
```

Protocols — duck typing, made checkable. Recall §1.6: Python doesn't care what class an object is, only whether it has the method you call (duck typing). A **Protocol**[11] lets you state that as a type — "anything with a `read()` returning bytes" — and have mypy check it, without the object inheriting anything:

```python
from typing import Protocol

class Readable(Protocol):
    def read(self) -> bytes: ...      # the required shape

def consume(source: Readable) -> None:    # accepts ANY object with read() -> bytes
    data = source.read()
```

This is **structural typing**[12]: compatibility is decided by an object's shape (its methods), not its declared class — in contrast to **nominal typing**[13], where it's decided by name/inheritance (an ABC, §1.7). A Protocol is the typed, checkable form of the duck typing you already met: same flexibility, now with a safety net.

5.5 Why a large codebase pays for typing

Annotations cost effort to write and maintain. A project the size of a node accepts that cost because the returns compound with scale:

- **Documentation that cannot rot.** A comment saying "returns a list of blocks" can drift out of date silently; a `-> list[Block]` annotation is verified every check, so it is always true.

- **Refactoring safety.** Change a function's return type and mypy instantly lists every caller that now breaks — turning a terrifying change into a checklist. This alone justifies typing in long-lived code.
- **Editor intelligence.** With types, an editor can offer accurate autocomplete[14] and catch mistakes as you type, because it knows what each variable is.
- **Bugs caught early.** The whole class of "wrong type" and "forgot the None case" errors is caught before the code runs.

The honest trade-offs: annotations add visual noise, occasionally fight you on genuinely-dynamic code (where `Any` or a cast becomes necessary), and require discipline to keep the checker passing. The consensus in large Python projects — and the choice `hathor-core` makes — is that the safety is worth the friction. Typing is a linter[15]-grade tool elevated to a contract.

5.6 Bridge — typing in `hathor-core`

The codebase is thoroughly annotated and checked. Forward-pointers; full treatment in the chapters named.

> **Bridge — typing in the codebase:**
>
> - **mypy in the toolchain.** mypy runs over the whole codebase as part of the quality gate, alongside flake8 and isort — §5.3 — **Chapter 20**.
> - **Domain type aliases.** `hathor/types.py` defines named aliases (e.g. for vertex IDs, addresses, amounts) so signatures read in domain terms rather than bare `bytes` / `int`; you'll see these everywhere in the model — §5.4 — **Chapter 25**.
> - **The zope plugin.** Because the node uses `zope.interface` (an interface system predating Protocols), mypy runs with the `mypy-zope` plugin so those interfaces are type-checked too — §5.4 / §1.7 — **Chapters 16 & 20**.
> - **Pydantic — where types act at run time.** `hathor-core` uses Pydantic for settings and some models. Pydantic reads the same type hints but, unlike plain Python, enforces them at run time — validating and converting incoming data. It is the one place the §5.2 "hints are ignored at run time" rule is deliberately overridden — §5.2 — **Chapters 18 & 22**.
> - **Protocols & ABCs for backends.** The interchangeable storage/index backends (the §1.7 contract) are expressed with abstract classes and checked by mypy, so a backend that misses a method fails the check, not production — §5.4 — **Chapters 27–28**.

Recap

Concept	One-line definition	Why it matters
Dynamic typing	Types checked at run time only	Flexible, but errors hide until executed
Type hint	An annotation declaring expected type	Self-checking documentation
Static typing	Types checked ahead of time, on source	Bugs caught before the code runs
mypy	Python's static type checker	Proves the hints hold across the codebase
`T | None` / Optional	"a value, or nothing"	Forces callers to handle the missing case
`Any`	"stop checking" escape hatch	Necessary sometimes; a hole in the net
Generic / `TypeVar`	One definition, type preserved	Reusable code without losing type info
Protocol	Required shape, checked structurally	Duck typing (§1.6) with a safety net
Pydantic	Hints enforced at run time	Validates external data using the same types

Type hints turn Python's run-time flexibility into a contract a tool can verify before the code ever runs. For a small script the discipline barely earns its keep; for a node it is load-bearing — it documents intent that cannot rot, makes large refactors safe, and catches a whole class of bugs at the keyboard instead of in production. You have now finished the programming-concepts track: objects (Ch 1), time and callbacks (Ch 2), the patterns that arrange them (Ch 3), the wrapping-and-routing machinery (Ch 4), and the typing discipline that holds it all together (Ch 5). The next chapters change subject entirely. Track B sets aside how Python works and takes up what a blockchain is — starting, in Chapter 6, with the problem of money without a bank.

---

1. Dynamic typing means variable types are not fixed and are checked while the program runs. A name can be bound to a value of any type, and type errors surface at execution time. Python, Ruby, and JavaScript are dynamically typed. ↩

2. A type annotation (type hint) is syntax attaching an expected type to a variable, parameter, or return value (`x: int`, `def f() -> str`). In Python it is informational by default — read by tools and humans, not enforced by the interpreter. ↩
3. Gradual typing allows mixing typed and untyped code in the same program, so annotations can be added incrementally rather than all at once. Python's type system is gradual. ↩
4. Static (in "static typing"/"static analysis") means "determined from the source ahead of run time," without executing the program. Its opposite is dynamic — determined while running. ↩
5. mypy is the standard static type checker for Python: a separate tool that reads type hints and reports inconsistencies without running the code. `hathor-core` runs it as part of its checks. ↩
6. Continuous integration (CI) is an automated pipeline that runs tests and checks (type-checking, linting) on every code change, so problems are caught before the change is merged. ↩
7. Static analysis is examining a program's source to learn facts about it (type errors, unused variables, security issues) without running it. Type-checking and linting are forms of static analysis. ↩
8. `Any` is the type that is compatible with everything and disables checking for the values it touches. It is the deliberate escape hatch from the type system; overusing it defeats the purpose of typing. ↩
9. A generic is a function or class parameterized by one or more types, so it works uniformly for many types while preserving them (e.g. a list that "remembers" it holds ints). ↩
10. A type variable (`TypeVar`) is a placeholder standing for "some specific type, the same throughout this signature," used to write generics. `def first(x: list[T]) -> T` returns the list's element type. ↩
11. A Protocol (PEP 544) defines a required set of methods/attributes; any object with that shape satisfies it, no inheritance needed. It is the statically-checkable form of duck typing. ↩
12. Structural typing decides type compatibility by an object's shape (which methods/attributes it has). "If it has the right methods, it fits." Protocols implement this. ↩
13. Nominal typing decides compatibility by declared name/inheritance — an object fits only if its class explicitly is (or subclasses) the required type. ABCs (§1.7) are nominal. ↩
14. Autocomplete is the editor feature that suggests valid attributes/methods as you type. Type information makes these suggestions accurate, because the editor knows each variable's type. ↩
15. A linter is a tool that flags suspicious or non-conforming code (style violations, likely bugs) by static analysis. flake8 is the linter in `hathor-core`'s toolchain; a type checker is a stricter cousin. ↩